

Comparando soma de vetores entre CPU e GPU em OpenMP

Eugenio Vitor Lopes dos Santos

DCA3703, Programação Paralela, Universidade Federal do Rio Grande do Norte

1 Enunciado

Fazer os exercícios de adição de vetores, `vadd.c`, dos slides 27 e 48 do tutorial de programação de GPUs com OpenMP em um dos nós com GPU do NPAD. Comparar os tempos de execução somente na CPU e com o uso da GPU. Reportar o progresso, apresentando os problemas encontrados e as soluções adotadas.

2 Fundamentação Teórica

- **Paralelismo de dados:** a soma de vetores aplica a mesma operação, $c_i = a_i + b_i$, a todos os elementos. Cada iteração escreve em uma posição distinta, permitindo executar o laço em paralelo sem dependência entre iterações.
- **CPU e GPU:** a CPU possui núcleos voltados à execução de tarefas variadas; a GPU explora muitas operações concorrentes sobre dados. Na soma de vetores, ambas podem distribuir os elementos entre suas unidades de execução, mas o uso da GPU acrescenta custos de lançamento e movimentação de dados.
- **#pragma omp parallel for:** cria uma região paralela e distribui as iterações de um laço entre as threads OpenMP da CPU. Neste experimento, é usada na inicialização, na soma da versão CPU e na verificação dos resultados.
- **#pragma omp target:** encaminha a execução de uma região para um dispositivo alvo. Os dados usados nessa região precisam estar disponíveis na memória do dispositivo, por meio de mapeamento explícito ou implícito.
- **Cláusula map:** controla a associação dos dados entre hospedeiro e dispositivo. O tipo `to` copia dados para o dispositivo; `from` copia os resultados para o hospedeiro; e `tofrom` permite cópias nos dois sentidos. Na implementação medida, os três vetores da soma receberam mapeamento implícito `tofrom`.
- **Diretivas teams e loop:** `teams` cria um conjunto de times de threads, enquanto `loop` indica que as iterações podem ser executadas concorrentemente e permite ao compilador organizar sua distribuição. A combinação `target teams loop` é usada para executar a soma na GPU.
- **Cláusula reduction:** mantém uma contribuição privada por thread e combina essas contribuições ao final. Na verificação, `reduction(+:err)` soma as contagens de erros sem atualizações concorrentes ao mesmo contador.

- **OMP_TARGET_OFFLOAD:** controla o comportamento das regiões destinadas ao dispositivo. O valor **MANDATORY** exige offload quando a região solicita execução no dispositivo. A consulta `omp_is_initial_device()` complementa essa configuração, indicando se a região executou no hospedeiro.
- **Transferência de dados e intensidade aritmética:** a soma realiza apenas uma adição por elemento, lendo duas entradas e escrevendo uma saída. Essa pequena quantidade de cálculo em relação aos dados movimentados pode tornar as transferências relevantes para o tempo total. Por isso, o tempo da região `target`, que inclui cópias e sincronização, deve ser distinguido do tempo isolado do kernel.

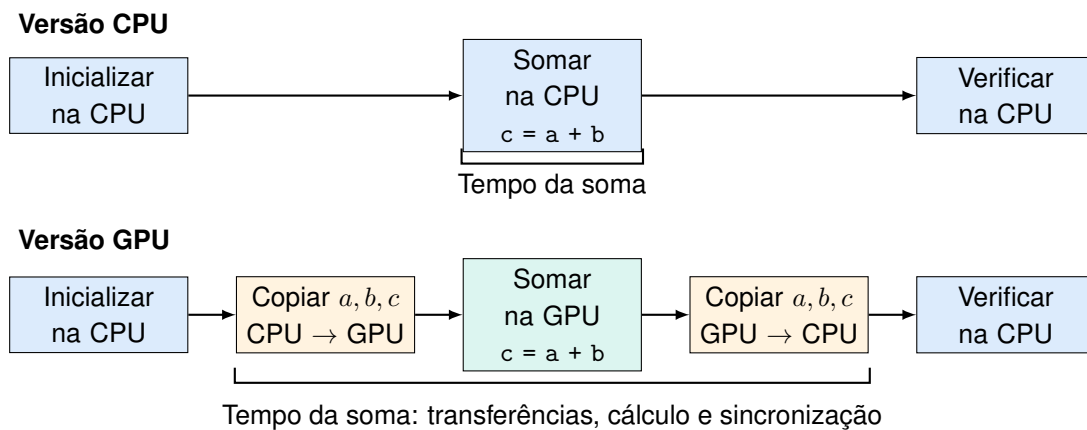


Figura 1: Etapas das duas versões, sem escala de tempo. O mapeamento implícito `tofrom` da implementação medida prevê cópias dos três vetores nos dois sentidos. Inicialização e verificação permanecem na CPU.

3 Experimento

3.1 Código comum e alteração entre versões

Os programas derivam do `vadd.c` do tutorial, escrito por Tim Mattson em 2017. Ambos usam $N = 10\,000\,000$ e quatro vetores de `float`: entradas `a` e `b`, saída `c` e referência `res`. São inicializados com `a[i] = (float)i`, `b[i] = 2.0*(float)i`, `c[i] = 0.0` e `res[i] = i + 2*i`.

A inicialização e a verificação permanecem na CPU nas duas versões. A verificação conta os elementos cujo erro ao quadrado excede 10^{-7} ; a cláusula `reduction(+:err)` combina as contagens das threads sem acesso concorrente ao mesmo contador.

Na versão CPU, o laço da soma usa:

```

1 #pragma omp parallel for
2 for (int i=0; i<N; i++) {
3     c[i] = a[i] + b[i];
4 }

```

Na versão GPU, somente a diretiva desse laço muda:

```

1 #pragma omp target teams loop

```

Antes dos intervalos medidos, a versão GPU executa uma região `target` que consulta `omp_is_initial_device()`. O programa encerra com erro se essa região executar na CPU. A configuração também define `OMP_TARGET_OFFLOAD=MANDATORY`. Essa região inicial confirma a execução no dispositivo e inicializa o runtime GPU; seu custo fica fora dos tempos apresentados.

3.2 Configuração e medição

O experimento foi executado no NPAD, na partição `gpu-8-v100`, com oito CPUs alocadas e uma GPU Tesla V100-SXM2 de 16 GB. O processador do nó era um Intel Xeon E5-2683 v4.

Cada versão foi executada cinco vezes, com $N = 10\,000\,000$ e oito threads na CPU. Foram usadas as afinidades `OMP_PLACES=cores` e `OMP_PROC_BIND=close`. A compilação utilizou NVIDIA HPC SDK 24.11 com otimização `-O3`:

```
1 nvc -O3 -mp=multicore vadd_cpu.c -o vadd_cpu
2 nvc -O3 -mp=gpu -Minfo=mp vadd_gpu.c -o vadd_gpu
```

Os tempos foram medidos com `clock_gettime(CLOCK_MONOTONIC)`, separando inicialização, soma e verificação. O total corresponde à soma desses intervalos. Na GPU, o tempo da soma inclui transferências e sincronização; a inicialização do runtime fica fora da medição. Os resultados apresentam a mediana das cinco execuções.

4 Progresso, problemas e soluções

A tentativa anterior, job 2129455, usou GCC e não reconheceu dispositivos OpenMP, registrando `na_gpu=0`. Apesar de produzir tempos e zero erros, essa execução não demonstrou uso da GPU. Seus arquivos foram preservados como registro do problema e seus tempos foram excluídos da comparação. A solução adotada foi carregar `compilers/nvidia/nvhpc/24.11`, compilar com `-mp=gpu` e exigir confirmação do dispositivo.

Os quatro vetores locais ocupam aproximadamente 160 MB na pilha. Para manter a alocação do código original, foi usado `ulimit -s unlimited` e `OMP_STACKSIZE=256M`. Não houve falha de pilha na execução medida.

5 Resultados

As dez execuções apresentaram zero erros. Em todas as cinco execuções da versão GPU, o programa detectou um dispositivo OpenMP e confirmou execução fora do hospedeiro. A Tabela 1 apresenta as medianas das cinco repetições por versão. O total é a mediana dos totais impressos, não a soma das medianas dos três intervalos.

Intervalo	CPU (s)	GPU (s)	Speedup
Inicialização	0,020963	0,020564	1,019
Soma	0,003680	0,041005	0,090
Verificação	0,002175	0,002007	1,084
Total dos intervalos	0,026641	0,064948	0,410

Tabela 1: Medianas de cinco execuções com $N = 10\,000\,000$. Na versão GPU, inicialização e verificação continuam na CPU. Speedup: $T_{\text{CPU}}/T_{\text{GPU}}$.

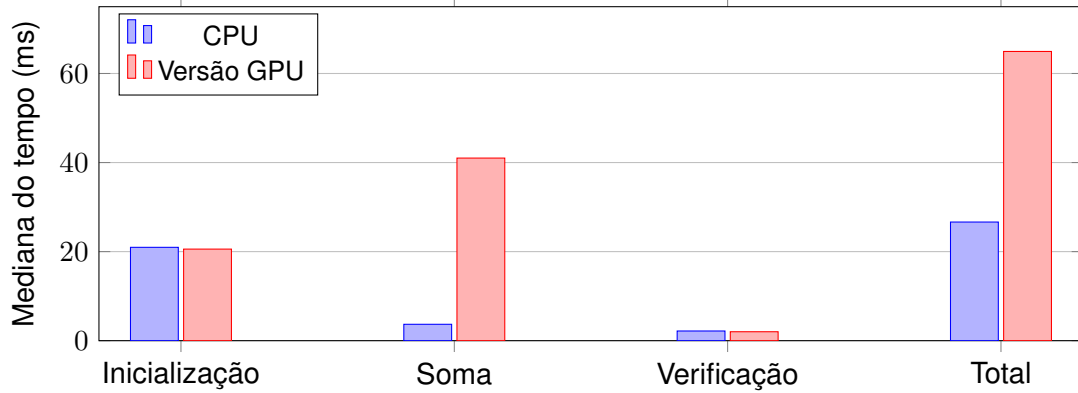


Figura 2: Comparação dos intervalos medidos. O tempo da soma na GPU inclui transferências e sincronização.

A CPU apresentou o menor tempo mediano de soma e de total. A soma na versão GPU demorou aproximadamente 11,1 vezes o tempo da CPU, enquanto o total dos intervalos foi aproximadamente 2,4 vezes maior. Inicialização e verificação ficaram próximas entre as versões, pois ambas executam essas etapas na CPU.

O resultado mostra que o offload de uma única soma, nesta implementação e carga, não compensou os custos incluídos na região `target`. O compilador informou mapeamento implícito `tofrom` para os três vetores. Cada vetor ocupa 40 MB; esse mapeamento prevê cópias de entrada e saída, inclusive para dados que a soma apenas lê ou apenas escreve. O tempo chamado de computação, portanto, também abrange movimentação de dados e não deve ser interpretado como duração isolada do kernel.

A baixa quantidade de cálculo por elemento e as transferências oferecem uma explicação plausível para o resultado, mas o experimento não mediu esses custos separadamente. Não é possível quantificar sua contribuição individual nem concluir que a GPU seja mais lenta em outras cargas ou em uma implementação que mantenha dados no dispositivo. Escalabilidade forte e fraca também não foram medidas.

Anexo A: Código CPU

```
1 #define _POSIX_C_SOURCE 199309L
2 #include <stdio.h>
3 #include <time.h>
4 #include <omp.h>
5 #define N 10000000
6 #define TOL 0.0000001
7 //
8 // This is a simple program to add two vectors
9 // and verify the results.
10 //
11 // History: Written by Tim Mattson, November 2017
12 //
13 static double agora(void)
14 {
15     struct timespec t;
16     clock_gettime(CLOCK_MONOTONIC, &t);
17     return t.tv_sec + t.tv_nsec * 1e-9;
18 }
19
20 int main(void)
21 {
22
23     float a[N], b[N], c[N], res[N];
24     int err=0;
25
26     double init_time, compute_time, test_time;
27     init_time = -agora();
28
29     // fill the arrays
30     #pragma omp parallel for
31     for (int i=0; i<N; i++){
32         a[i] = (float)i;
33         b[i] = 2.0*(float)i;
34         c[i] = 0.0;
35         res[i] = i + 2*i;
36     }
37
38     init_time += agora();
39     compute_time = -agora();
40
41     // add two vectors
42     #pragma omp parallel for
43     for (int i=0; i<N; i++){
44         c[i] = a[i] + b[i];
45     }
46
47     compute_time += agora();
48     test_time = -agora();
49
50     // test results
51     #pragma omp parallel for reduction(+:err)
```

```

52  for(int i=0;i<N;i++){
53      float val = c[i] - res[i];
54      val = val*val;
55      if(val>TOL) err++;
56  }
57
58  test_time    +=  agora();
59
60  printf(" vectors added with %d errors\n",err);
61
62  printf("Init time:      %.6fs\n", init_time);
63  printf("Compute time:  %.6fs\n", compute_time);
64  printf("Test time:     %.6fs\n", test_time);
65  printf("Total time:    %.6fs\n", init_time + compute_time + test_time);
66  return err != 0;
67 }

```

Anexo B: Código GPU

```
1 #define _POSIX_C_SOURCE 199309L
2 #include <stdio.h>
3 #include <time.h>
4 #include <omp.h>
5 #define N 10000000
6 #define TOL 0.0000001
7 //
8 // This is a simple program to add two vectors
9 // and verify the results.
10 //
11 // History: Written by Tim Mattson, November 2017
12 //
13 static double agora(void)
14 {
15     struct timespec t;
16     clock_gettime(CLOCK_MONOTONIC, &t);
17     return t.tv_sec + t.tv_nsec * 1e-9;
18 }
19
20 int main(void)
21 {
22
23     float a[N], b[N], c[N], res[N];
24     int err=0;
25
26     int no_host = 1;
27     #pragma omp target map(from:no_host)
28     { no_host = omp_is_initial_device(); }
29     if (no_host) {
30         fprintf(stderr, "Erro: a regio target executou na CPU.\n");
31         return 1;
32     }
33     printf("Dispositivos OpenMP: %d; target executou na GPU\n", omp_get_num_devices
           ());
34
35     double init_time, compute_time, test_time;
36     init_time = -agora();
37
38     // fill the arrays
39     #pragma omp parallel for
40     for (int i=0; i<N; i++){
41         a[i] = (float)i;
42         b[i] = 2.0*(float)i;
43         c[i] = 0.0;
44         res[i] = i + 2*i;
45     }
46
47     init_time += agora();
48     compute_time = -agora();
49
50     // add two vectors
```

```

51  #pragma omp target teams loop
52  for (int i=0; i<N; i++){
53      c[i] = a[i] + b[i];
54  }
55
56  compute_time +=  agora();
57  test_time     = -agora();
58
59  // test results
60  #pragma omp parallel for reduction(+:err)
61  for(int i=0; i<N; i++){
62      float val = c[i] - res[i];
63      val = val*val;
64      if(val>TOL) err++;
65  }
66
67  test_time     +=  agora();
68
69  printf(" vectors added with %d errors\n",err);
70
71  printf("Init time:      %.6fs\n", init_time);
72  printf("Compute time:  %.6fs\n", compute_time);
73  printf("Test time:     %.6fs\n", test_time);
74  printf("Total time:    %.6fs\n", init_time + compute_time + test_time);
75  return err != 0;
76 }

```